Lecture 4: Dense Retrieval and Embedding Representations

Slide 1: Bi-Encoders and Dense Vector Spaces

Unlike sparse keyword search (TF-IDF, BM25), dense retrieval maps textual queries and passages into continuous vector spaces where semantic similarity corresponds to vector proximity. Dense bi-encoders encode query and document independently, allowing pre-computation and indexing of document embeddings.

Our course standard embedding model is sentence-transformers/all-MiniLM-L6-v2, which maps text into 384-dimensional dense vectors. It provides high inference speed and competitive semantic retrieval benchmarks on MTEB.

Slide 2: Document Chunking Strategies

Optimal retrieval requires balanced chunking. If chunks are too small (e.g. 100 characters), they lack contextual coherence. If chunks are too large (>1500 characters), embedding vectors become diluted with tangential topics.

We configure RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=80 characters. The overlap ensures boundary sentences are not severed across splits, preserving contiguous semantic meaning.

Slide 3: FAISS Vector Indexing (IndexFlatL2)

Facebook AI Similarity Search (FAISS) enables ultra-fast vector nearest neighbor search. In our curriculum, we utilize faiss.IndexFlatL2, which conducts an exhaustive, exact Euclidean distance search. For small to medium knowledge bases (<100,000 vectors), IndexFlatL2 guarantees 100% recall without indexing distortion. Larger deployments migrate to Inverted File Indexing (IndexIVFFlat) or HNSW for sub-linear search complexity.

Slide 4: Grounded Generation and Hallucination Mitigation

RAG bridges the parametric knowledge gap by injecting retrieved chunks into the prompt context window. Prompt instructions explicitly command the model: 'Answer strictly using the provided context. If the answer is missing, output INSUFFICIENT_CONTEXT.' This prevents hallucinations and bounds generation to verifiable course facts.